

Week 2 - Day 8a: More Recursion

Exercise 1: Power of Two []

Power of Two is a fundamental concept in computer science and mathematics. A number is a power of two if it can be written as 2^n for some non-negative integer n .

The first few powers of two are:

- $2^0 = 1$
- $2^1 = 2$
- $2^2 = 4$
- $2^3 = 8$
- $2^4 = 16$
- $2^5 = 32$
- $2^6 = 64$
- $2^7 = 128$

Mathematically, we can write the n -th power of two, $P(n)$, using a recurrence relation:

$$P(0) = 1$$

and for $n \geq 1$ we have

$$P(n) = 2 \cdot P(n-1)$$

In Week 1, we saw how to implement this sequence **iteratively** using while-loops. This time, let's try to implement `power_of_two` **recursively**.

Task: Write a recursive function `power_of_two(n)` that returns the n -th power of two (i.e., 2^n).

Examples:

```
power_of_two(0) == 1 # 2^0 = 1
power_of_two(1) == 2 # 2^1 = 2
power_of_two(2) == 4 # 2^2 = 4
power_of_two(3) == 8 # 2^3 = 8
power_of_two(4) == 16 # 2^4 = 16
power_of_two(5) == 32 # 2^5 = 32
power_of_two(6) == 64 # 2^6 = 64
power_of_two(7) == 128 # 2^7 = 128
```

```
# TODO: Implement a recursive function to find the n-th power of two
def power_of_two(n):
```

```

    # your code here
    pass

# Test cases
assert power_of_two(0) == 1
assert power_of_two(1) == 2
assert power_of_two(2) == 4
assert power_of_two(3) == 8
assert power_of_two(4) == 16
assert power_of_two(5) == 32
assert power_of_two(6) == 64
assert power_of_two(7) == 128

print("All tests passed. ☑")

```

Exercise 2: Tribonacci Numbers []

Tribonacci numbers are similar to Fibonacci numbers but compute the next element as the sum of the previous three elements. The first three terms are defined as:

$$T(0)=0, T(1)=0, T(2)=1$$

and for $n \geq 3$ the n -th Tribonacci number is recursively defined as:

$$T(n)=T(n-1)+T(n-2)+T(n-3)$$

Task: Implement `tribonacci` which returns the n -th Tribonacci number recursively.

```

# TODO: Implement a recursive function to find the n-th Tribonacci
number
def tribonacci(n):
    # your code here
    pass

# Here are some test cases to test your solution
assert tribonacci(0) == 0
assert tribonacci(1) == 0
assert tribonacci(2) == 1
assert tribonacci(3) == 1
assert tribonacci(4) == 2
assert tribonacci(5) == 4
assert tribonacci(11) == 149

print("All tests passed. ☑")

```

Exercise 3: Nested List Traversal []

Implement a function `print_list` which prints out all the elements of a nested list.

For example, if we have

```
lst = [["hello"], "darkness", ["my", "old", ["friend"]],
```

it should print:

```
hello darkness my old friend
```

```
# TODO: Implement a function to print all elements of a nested list
def print_list(lst):
    # your code here
    pass

# Use this cell to test your function
print_list(["hello", "darkness", ["my", "old", ["friend"]]])
```

Exercise 4: Nested List Sum []

Similar to the previous exercise, we can also sum all the integers in a nested list. Implement a recursive function called `nested_list_sum` which takes a nested list `lst`, which can contain either integers or nested lists. The function returns the sum of all the integers in the nested list.

Examples:

```
nested_list_sum([1, 2, 3, [4, 5]]) == 15
nested_list_sum([[1, 2], [3, [4, 5]]]) == 15
nested_list_sum([1, [2, [3, [4, [5]]]]) == 15

# TODO: Implement a recursive function to sum all integers in a nested list
def nested_list_sum(lst):
    # your code here
    pass

# Here are some test cases to test your solution
assert nested_list_sum([]) == 0
assert nested_list_sum([1, 2, 3, [4, 5]]) == 15
assert nested_list_sum([[1, 2], [3, [4, 5]]]) == 15
assert nested_list_sum([1, [2, [3, [4, [5]]]]) == 15

print("All tests passed. [ ]")
```

Exercise 5: Catalan Numbers [] []

The [Catalan numbers](#) are a sequence of natural numbers that occur in many different counting problems.

They are defined recursively as follows:

$$C_0=1$$

$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}$$

For example,

$$C_1 = C_0 \cdot C_0$$

$$C_2 = C_0 \cdot C_1 + C_1 \cdot C_0$$

$$C_3 = C_0 \cdot C_2 + C_1 \cdot C_1 + C_2 \cdot C_0$$

$$C_4 = C_0 \cdot C_3 + C_1 \cdot C_2 + C_2 \cdot C_1 + C_3 \cdot C_0$$

Implement `catalan` which computes C_n given an input value `n` recursively.

```
# TODO: Implement a recursive function to compute the n-th Catalan
number
def catalan(n):
    # your code here
    pass

# Here are some test cases to test your solution
assert catalan(0) == 1
assert catalan(1) == 1
assert catalan(2) == 2
assert catalan(3) == 5
assert catalan(5) == 42
assert catalan(7) == 429

print("All tests passed. 🎉")
```

One useful application of Catalan numbers is to count the number of correctly matched parentheses expressions. For a sequence of parentheses expressions to be correctly matched, every opening parenthesis (must have a corresponding closing parenthesis). The n -th Catalan number counts the number of valid sequences that have n pairs of parentheses. For example:

- For $C_1=1$, the only way to arrange $n=1$ pairs of parentheses is `()`.
- For $C_2=2$, we can arrange $n=2$ pairs of parentheses by `(( ))` and `() ()`.
- For $C_3=5$, we can arrange $n=3$ pairs of parentheses in the following combinations: `(( ( ))`, `(( ) )`, `(( )) ()`, `() (( ))`, `() () ()`.

Now you see how powerful Catalan numbers (and recursion) are!

Exercise 6: Harmonic Sum []]

Implement a function `harmonic_sum` which computes the harmonic sum up to the n -th element. The harmonic sum is the sum of reciprocals of the positive integers. For example,

```

harmonic_sum(1) == 1
harmonic_sum(2) == (1 + 1/2)
harmonic_sum(3) == (1 + 1/2 + 1/3)
harmonic_sum(4) == (1 + 1/2 + 1/3 + 1/4)

```

Task: Implement `harmonic_sum(n)` to compute the n -th harmonic sum.

```

# TODO: Implement a function to compute the harmonic sum up to the n-
# th element
def harmonic_sum(n):
    # your code here
    pass

# Here are some test cases to test your solution
assert harmonic_sum(1) == 1
assert harmonic_sum(2) == 1 + 1 / 2
assert harmonic_sum(3) == 1 + 1 / 2 + 1 / 3
assert harmonic_sum(4) == 1 + 1 / 2 + 1 / 3 + 1 / 4
assert harmonic_sum(5) == 1 + 1 / 2 + 1 / 3 + 1 / 4 + 1 / 5
assert int(harmonic_sum(1000) * 10**6) == 7485470

print("All tests passed. ☑")

```

Exercise 7: Robot Grid [] []

Write a function, `robot_grid(m, n)` that calculates the **number of unique paths** a robot placed at the top-left corner of an $m \times n$ grid can take to reach the bottom-right corner. On every move, the robot can only move down or right.

Examples:

- `robot_grid(0, 0) == 1`, because there is only one way to reach the bottom-right corner of a 0×0 grid.
- `robot_grid(1, 1) == 2`, because there are two ways to reach the bottom-right corner of a 1×1 grid. (Right -> Down, Down -> Right)
- `robot_grid(2, 2) == 6`, because there are six ways to reach the bottom-right corner of a 2×2 grid.

Illustration:

Robot Grid

Task: Implement `robot_grid(m, n)` recursively.

```

# TODO: Implement a recursive function to compute the number of unique
# paths in an m x n grid
def robot_grid(m, n):
    # your code here
    pass

```

```
# Here are some test cases to test your solution
assert robot_grid(0, 0) == 1
assert robot_grid(1, 1) == 2
assert robot_grid(1, 2) == 3
assert robot_grid(2, 2) == 6
assert robot_grid(3, 2) == 10

print("All tests passed. ☑")
```

Exercise 8: Flood Fill [□□□]

Finally, we will apply recursion to a more complex problem involving images. Without getting into the details of the library, try to run the below code cell to generate a random image of square size n , where p is the probability of a pixel being black.

```
# Run this cell to install the required dependencies otherwise the
next cell will fail
%pip install pillow

import random
from simpleimage import SimpleImage

def get_random_image(n, p):
    image = SimpleImage.blank(n, n)
    for y in range(n):
        for x in range(n):
            if random.random() <= p:
                image.set_rgb(x, y, 0, 0, 0)

    # Set centre pixel to black
    image.set_rgb(n // 2, n // 2, 0, 0, 0)

    return image

image = get_random_image(15, 0.7)
image.show(resize_width=200)
```

Now, given a black and white image, we want to implement a recursive function `floodfill` that fills all the connected black pixels with a different color. The function should take the image and the coordinates of a black pixel as input and return the modified image.

For example, if we do flood-fill with red color on the center of the below image, the output should look like the right image.

Implement `floodfill(img, x, y, r, g, b)` function below. Its parameters are an image `img` to do the flood-fill on, a starting position `x` and `y`, and the `r`, `g`, and `b` components are the color values to fill the region with.

```
# TODO: Implement a recursive function for flood-fill
def floodfill(img, x, y, r, g, b):
    # your code here
    pass

# Use this code to test your solution
n, p = 15, 0.7
image = get_random_image(n, p)
image.show(resize_width=200)

# Flood fill
floodfill(image, n // 2, n // 2, 255, 0, 0)
image.show(resize_width=200)
```

Congratulations 🎉